Docker - это программа для упрощения разработки, запуска и распространения ПО.

Докер решает следующие проблемы:

- контейнеризация приложений и их частей решает проблему переплетения зависимостей и запросов к библиотекам и тп от разных частей проекта
- каждый контейнер изолирует свое окружение, где находятся свои версии всех нужных библиотек
- кроссплатформенность: контейнеры можно запустить и на винде, и на маке, и на линуксе. Дело в том, что код пишется для условной виртуалки, а дальше он реализуется посредством среды разработки, то есть самого докера, который доступен как раз таки на любой системе
- развертывание сервисов в связке с Kubernetes: мы можем разворачивать и контролировать много сервисов сразу. В таком случае все описывается конфигурационными файлами

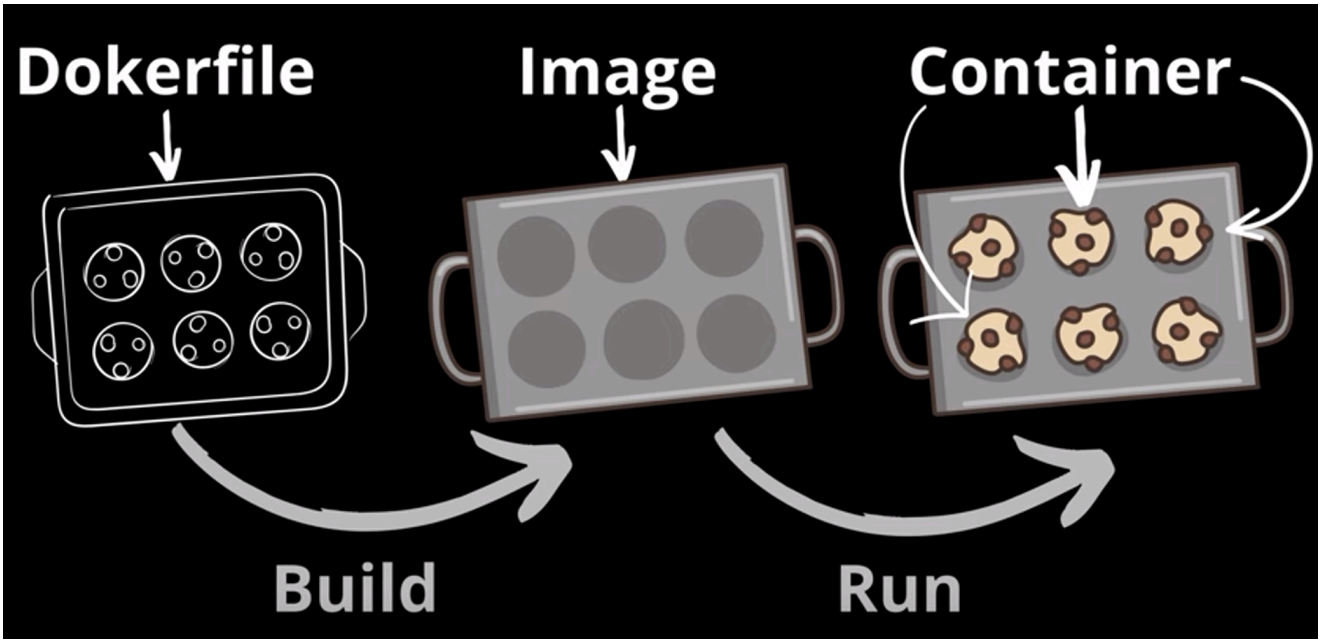
Основные понятия

Docker Engine - клиент-серверное приложение, состоящее из сервера, создающего контейнеры, образы и тома и управляющего ими, API для взаимодействия с сервером и CLI (нужен для взаимодействия с сервером через апи).

Основные компоненты Docker

Docker File - инструкции как создать образ.
Image - шаблон для контейнеров.
Контейнер - приложение на основе Image, исполняющееся в закрытой файловой системе со своим пространством процессов.

Примерно так это выглядит на примере с приготовлением печенек:



Установка Docker

Удобнее всего использовать Docker Desktop, который включает в себя все необходимое для работы.
Установка сама по себе такая же, как у любой программы.
После установки можно выполнить проверку по команду docker -v.

```
PS C:\Users\Влад> docker -v
Docker version 28.3.0, build 38b7060
```

Базовые команды

docker run ubuntu - запуск контейнера, в данном случае ubuntu

```
PS C:\Users\Влад> docker -v
Docker version 28.3.0, build 38b7060
PS C:\Users\Влад> docker run ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
b08e2ff4391e: Pull complete
Digest: sha256:440dcf6a5640b2ae5c77724e68787a906afb8ddee98bf86db94eea8528c2c076
Status: Downloaded newer image for ubuntu:latest
PS C:\Users\Влад>
```

Как видно на рисунке выше, контейнер запустился и сразу закончил работу. Это произошло потому что в контейнере нет никакого приложения.
Используя опцию -it можно запустить контейнер в интерактивном режиме.

```
PS C:\Users\Влад> docker run -it ubuntu

root@48a15109912c:/#
root@48a15109912c:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@48a15109912c:/#
```

Как видно на рисунке выше, мы запустились на ubuntu через контейнер и выполнили команду ls.

При помощи команды exit можно выйти из контейнера.

```
root@48a15109912c:/# exit
exit
PS C:\Users\Влад>
```

Чтобы посмотреть работающие контейнеры можно использовать команду docker ps.

```
PS C:\Users\Влад> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\Users\Влад>
```

Как видно, сейчас нет работающих контейнеров.

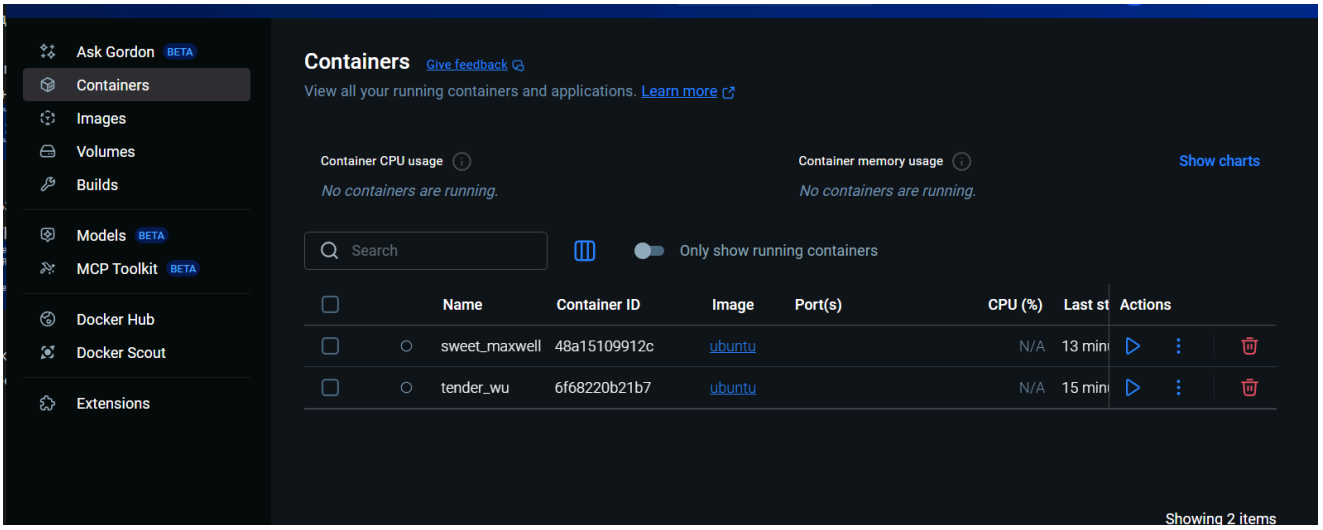
При помощи опции -a можно посмотреть все контейнеры, которые в принципе есть в системе.

```
PS C:\Users\Влад> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED        STATUS                        PORTS     NAMES
48a15109912c   ubuntu    "/bin/bash"   3 minutes ago  Exited (0) About a minute ago           sweet_maxwell
6f68220b21b7   ubuntu    "/bin/bash"   4 minutes ago  Exited (0) 4 minutes ago                tender_wu
PS C:\Users\Влад>
```

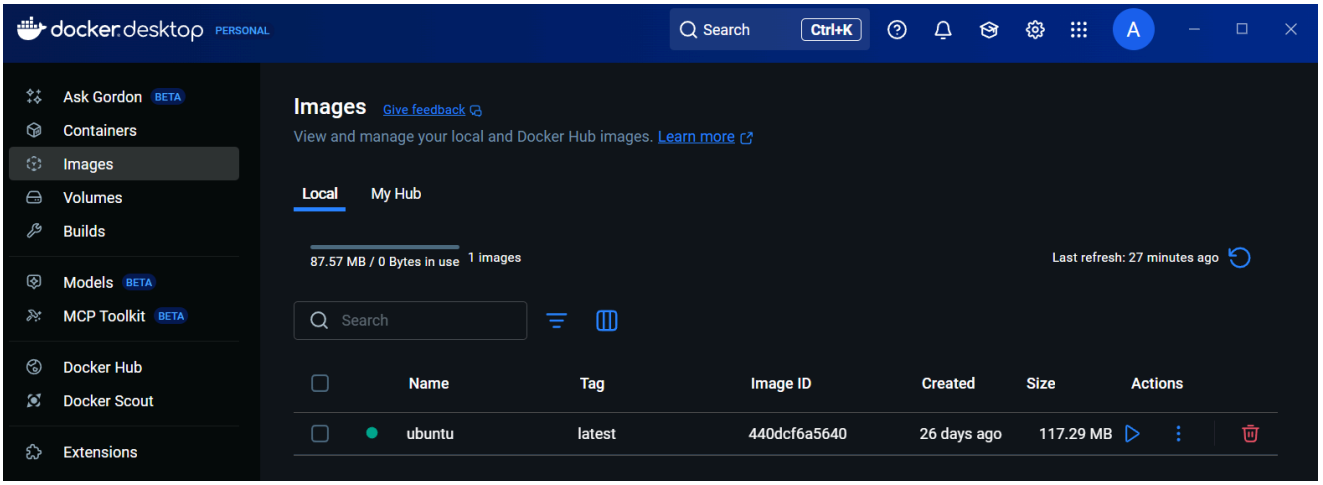
Используя команду docker images можно посмотреть все образы.

```
PS C:\Users\Влад> docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
ubuntu       latest    440dcf6a5640   3 weeks ago   117MB
PS C:\Users\Влад>
```

Все тоже самое можно сделать через графический интерфейс.
Во вкладке containers видно 2 текущих контейнера.



Во вкладке images мы видим тот самый образ.



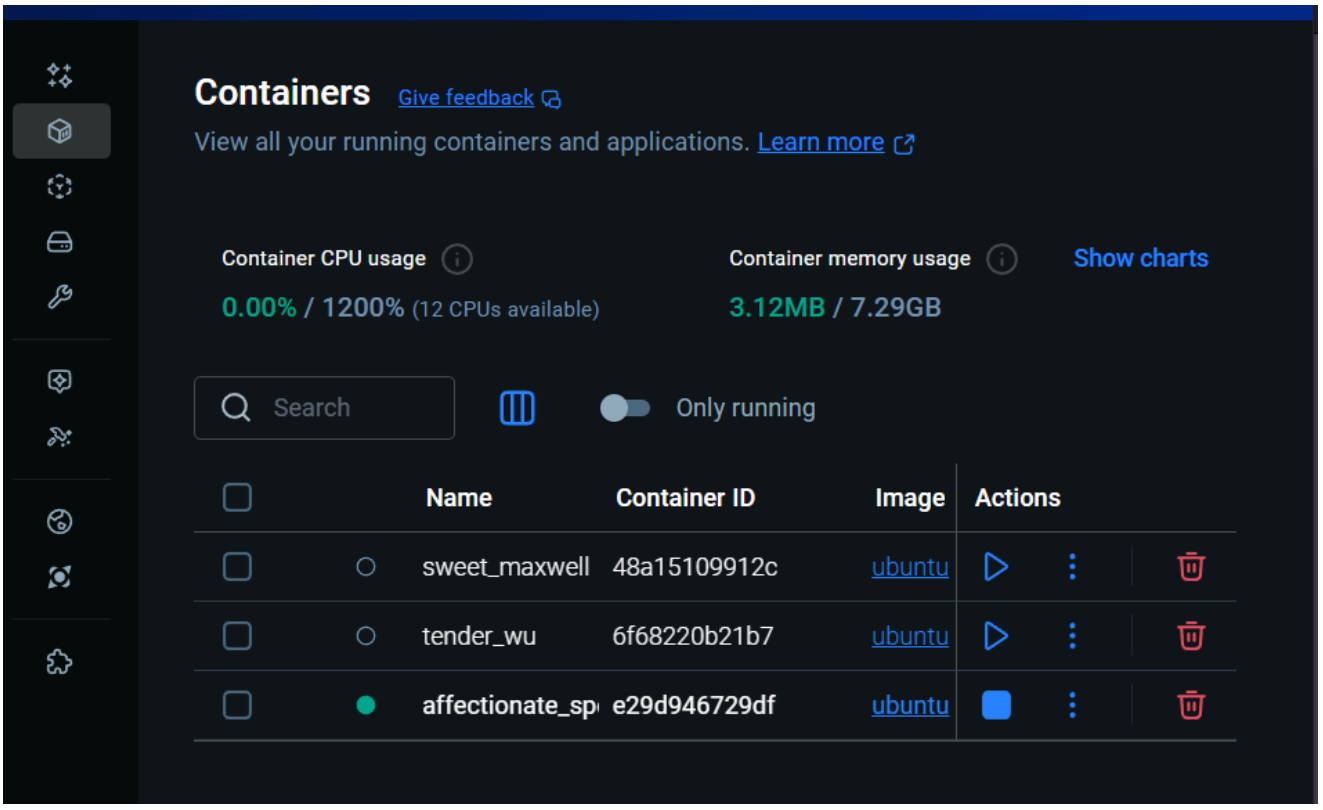
Для запуска контейнера в detouch-режиме можно использовать команду docker run -d -it ubuntu.

```
PS C:\Users\Влад> docker run -d -it ubuntu
e29d946729dff74009b4070a00b47c33ca20675a805ec4999f1ce87bd175
4ad8
PS C:\Users\Влад>
```

В таком случае терминал через контейнер откроется не сразу. Но теперь, если ввести команду ps, то мы увидим, что контейнер работает.

```
PS C:\Users\Влад> docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED
STATUS              PORTS     NAMES
e29d946729df   ubuntu    "/bin/bash"   About a minute ago
Up About a minute           affectionate_spence
PS C:\Users\Влад>
```

Аналогично будет, если посмотреть в графическом интерфейсе:



Здесь, в графическом интерфейсе, зеленым будут отображаться только работающие контейнеры, а остановленные - серым.

Теперь для того, чтобы присоединиться к работающему контейнеру нужно использовать команду docker attach, приписав к ней идентификатор нашего контейнера.

```
PS C:\Users\Влад> docker attach e29d946729dff74009b4070a00b4
7c33ca20675a805ec4999f1ce87bd1754ad8
root@e29d946729df:/# ls
bin   dev  home  lib64  mnt  proc  run   srv  tmp  var
boot  etc  lib   media  opt  root  sbin  sys  usr
root@e29d946729df:/#
```

Чтобы выйти из контейнера, не останавливая его, можно использовать сочетания клавиш Ctrl+P и Ctrl+Q.

```
root@e29d946729df:/# read escape sequence
PS C:\Users\Влад>
```

в моем случае работает только Ctrl+Q

Как и с запуском, можно интуитивной командой остановить контейнер:

```
PS C:\Users\Влад> docker stop e29d946729dff74009b4070a00b47c33c
a20675a805ec4999f1ce87bd1754ad8
e29d946729dff74009b4070a00b47c33ca20675a805ec4999f1ce87bd1754ad
8
PS C:\Users\Влад> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS
  NAMES
PS C:\Users\Влад>
```

Вновь запустить контейнер мы можем по команде start.

```
PS C:\Users\Влад> docker start e29d946729dff74009b4070a00b47c33
ca20675a805ec4999f1ce87bd1754ad8
e29d946729dff74009b4070a00b47c33ca20675a805ec4999f1ce87bd1754ad
8
PS C:\Users\Влад> docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED         STATUS
        PORTS     NAMES
e29d946729df   ubuntu    "/bin/bash"   9 minutes ago   Up 15 se
conds             affectionate_spence
PS C:\Users\Влад>
```

Работа с портами

Попробуем запустить сервер nginx, который, предположим, понадобился нам для разработки чего-то.

Запустим его через докер.

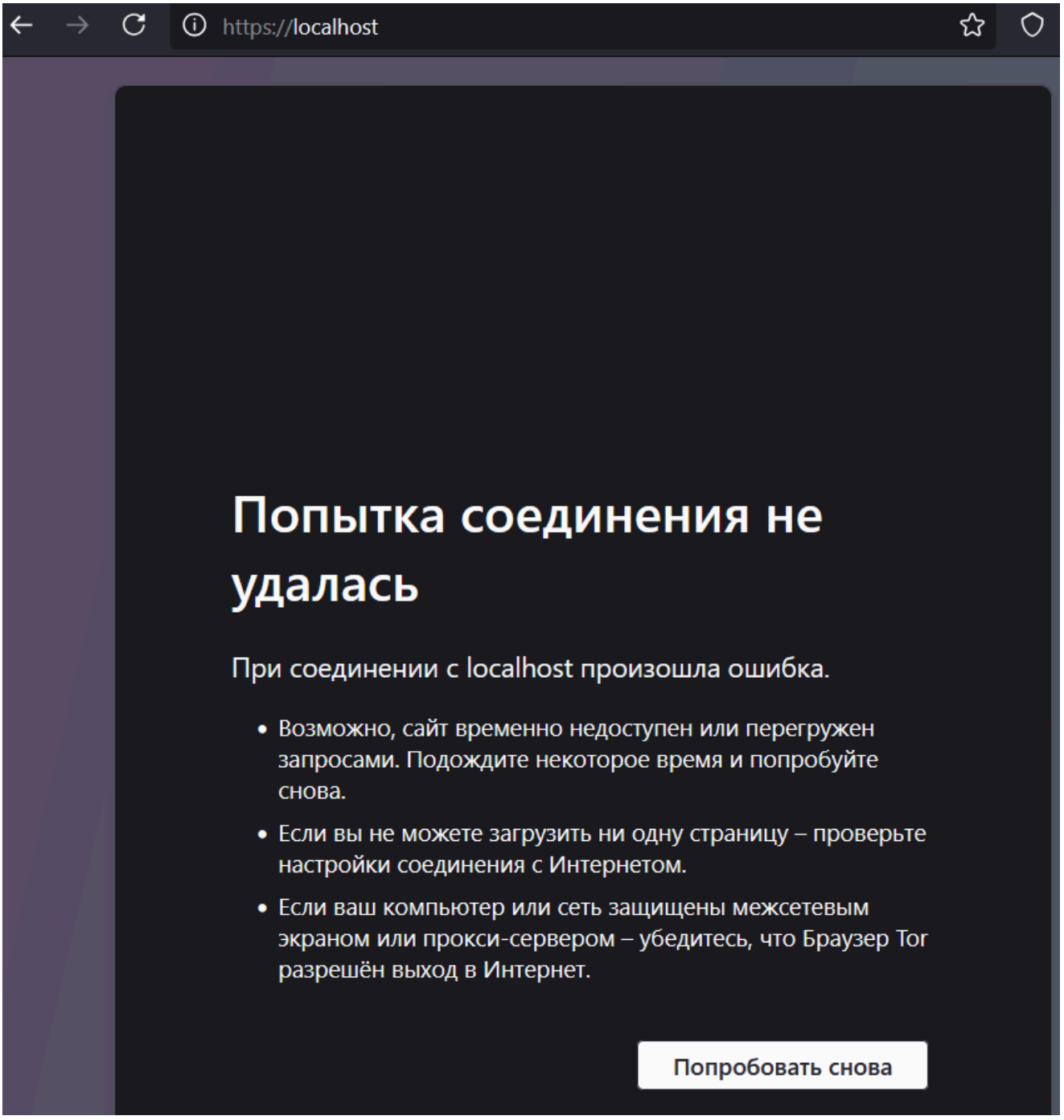
```
PS C:\Users\Влад> docker run -d nginx
290e9695e663a700a3c0f6ff82b5ae5fff28a9205aa65e1a0366bf34187d
c786
PS C:\Users\Влад>
```

Проверим, действительно ли он запущен:

```
PS C:\Users\Влад> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED
           STATUS              PORTS     NAMES
3e26a52ffe66   ubuntu    "/bin/bash"              4 seconds
ago        Up 3 seconds                   zen_goldstine
290e9695e663   nginx     "/docker-entrypoint.…"   About a mi
nute ago   Up About a minute   80/tcp    friendly_chaplygin
PS C:\Users\Влад>
```

Видно, что nginx используем порт 80. Попробуем обратиться к нему, постучавшись на

этот порт через браузер.



Видим, что страница недоступна. Это связано с тем, что nginx запущен в контейнере, который изолирован, а доступа извне к нему нет, поскольку этот порт закрыт сейчас.

Открытые порты - основной путь взаимодействия между портами.

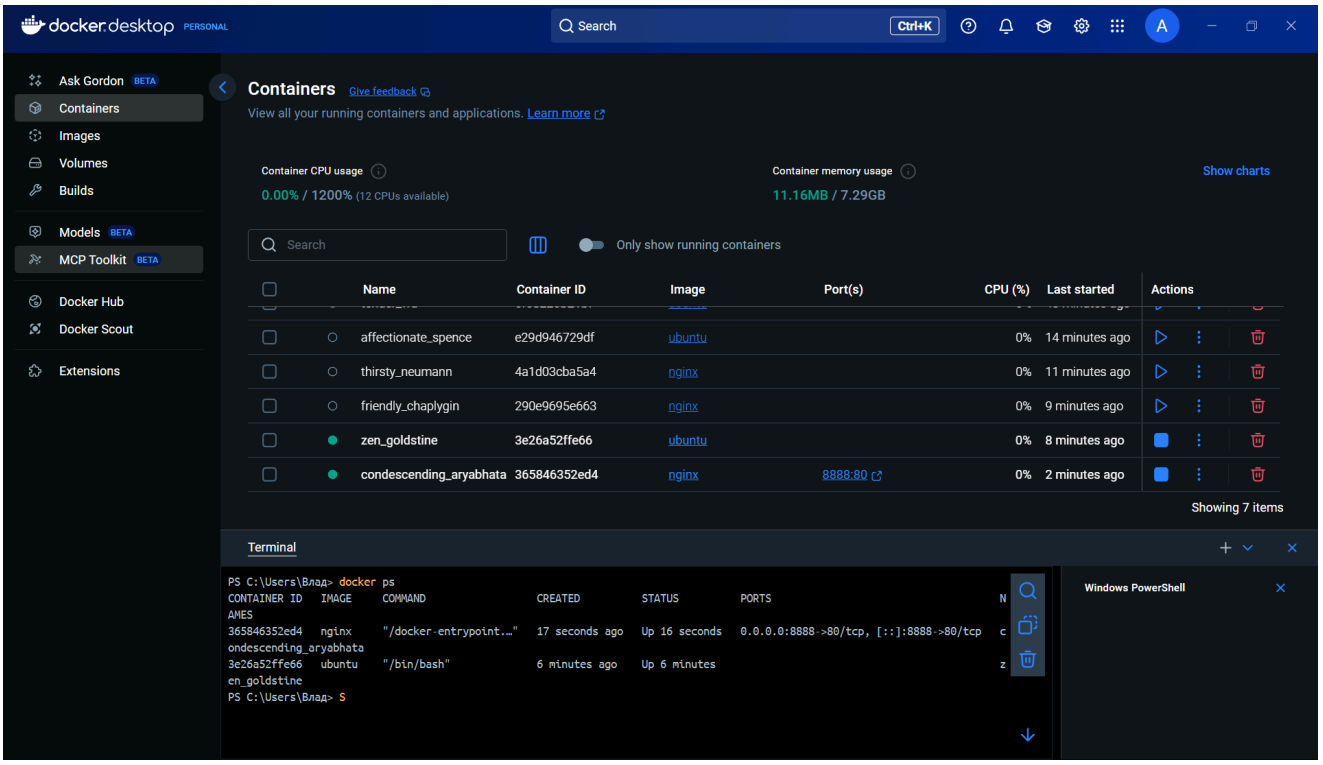
Теперь остановим этот контейнер.

```
PS C:\Users\Влад> docker stop 290e9695e663a700a3c0f6ff82b5ae
5fff28a9205aa65e1a0366bf34187dc786
290e9695e663a700a3c0f6ff82b5ae5fff28a9205aa65e1a0366bf34187d
c786
PS C:\Users\Влад>
```

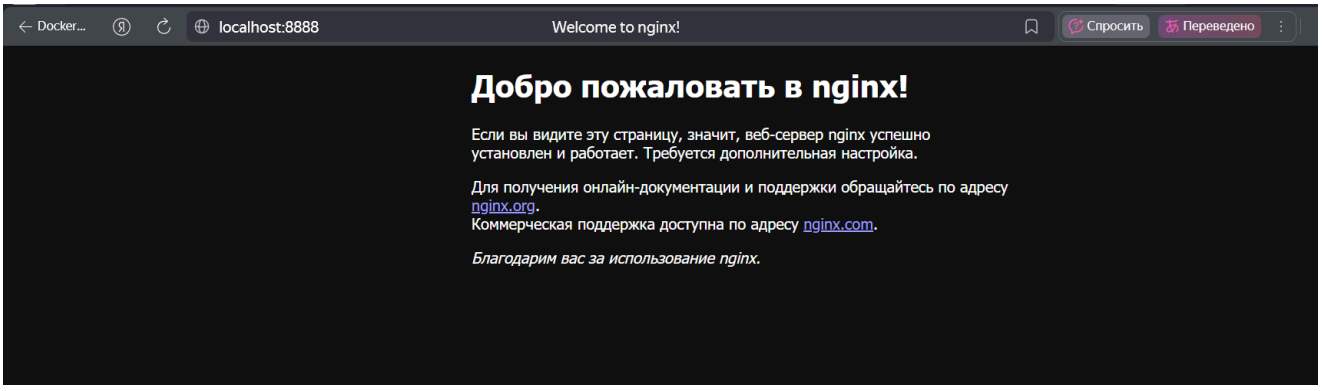
Чтобы запустить контейнер с открытым портом надо использовать флаг -p и указать порт, который будет переадресован в открытый в контейнере порт 80. Для примера

пусть это будет порт 8888.

Теперь мы видим, что отображается не только порт 80, но и порт 8888, на который сделана переадресация.



Теперь, перейдя в браузере по localhost на этот порт, страница загрузится:



Создание образов

Попробуем создать собственный образ на базе nginx.

```
PS C:\Users\Влад> mkdir docker_nginx


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        15.07.2025     17:28                docker_nginx
```

Перейдем в созданную директорию.

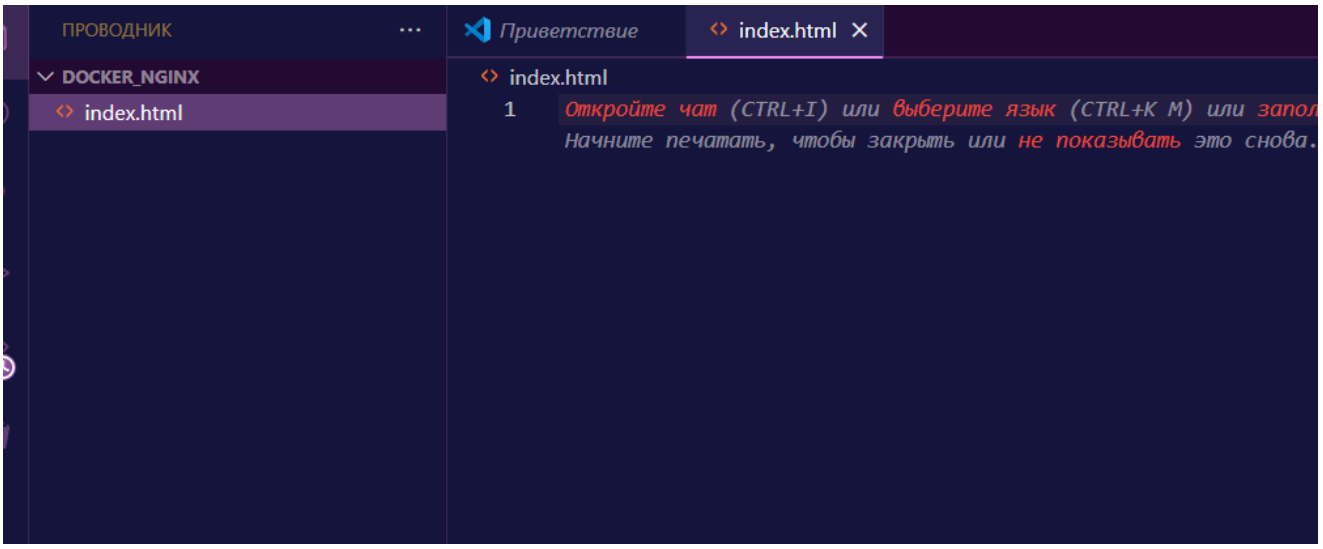
```
PS C:\Users\Влад> cd docker_nginx
PS C:\Users\Влад\docker_nginx>
```

Перейдем в редактор кода.

```
PS C:\Users\Влад\docker_nginx> code .
PS C:\Users\Влад\docker_nginx>
```

Далее откроется VS Code.

Создадим файл index.html и напишем в нем код.




```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewpoint" content="width=device-width, ">
    <title>Hello from docker</title>
    <style>
        body {
            text-align: center;
            font-family: Arial, Helvetica, sans-serif;
        }
    </style>
</head>
<img>
    <h1>Hello World from nginx in docker</h1>
    <img
src="https://camo.githubusercontent.com/180150afc7bc9d050ae2b56b884741f52b6489ebc83495cdb
94f457af4b5a819/68747470733a2f2f6c6f676f732d776f726c642e6e65742f77702d636f6e74656e742f757
06c6f6164732f323032312f30322f446f636b65722d53796d626f6c2e706e67"></img>

</body>
</html>
```

Теперь надо это упаковать в контейнер. Для этого создадим Dockerfile (у него нет никаких расширений).

В этом файле надо указать что берется за базу, то есть в нашем случае это nginx.

Далее указываем рабочую директорию и копируем в этот файл index.html.

```
ПРОВОДНИК
DOCKER_NGINX
Dockerfile
index.html
```

```
Dockerfile > ...
1  FROM nginx
2  WORKDIR /usr/share/nginx
3  COPY index.html .
```

Затем вернемся к консоли (можно в ней работать как из GUI докера, так и в PowerShell по идее).

Создадим образ на основе этого файла.

```
Run 'docker --help' for more information
PS C:\Users\Влад\docker_nginx> docker build . -t nginx-hello-world
[+] Building 1.0s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 90B
```

Теперь побегут логи построения образа.

```
Run 'docker --help' for more information
PS C:\Users\Влад\docker_nginx> docker builds . t nginx-hello-world
 => [2/3] WORKDIR /usr/share/nginx                                                    0.0s
 => [3/3] COPY index.html .                                                           0.1s
 => exporting to image                                                                0.3s
 => => exporting layers                                                               0.1s
 => => exporting manifest sha256:88819ad927d8a890224c020fa0158a1c166f6d2bfeee2c68d1f0e68df5a12ac7  0.0s
 => => exporting config sha256:1c91ba32808bc35010b78070969ec4374f7579137c622f90e58a3bb2f9859b30  0.0s
 => => exporting attestation manifest sha256:1552dd83db23fa1bb4142d79f41cd19428b8a82dd5f5bc3132ee1661afa0d7e9  0.0s
 => => exporting manifest list sha256:48d55fb90cc8dfb7b4d8e2f1cc6f051c5d6a579e50c458baf2c80f638aaadde5  0.0s
 => => naming to docker.io/library/nginx-hello-world:latest                           0.0s
 => => unpacking to docker.io/library/nginx-hello-world:latest                        0.1s
PS C:\Users\Влад\docker_nginx>
```

Теперь посмотрим на текущий список образов:

```
 => => unpacking to docker.io/library/nginx-hello-world:latest
PS C:\Users\Влад\docker_nginx> docker images
REPOSITORY          TAG       IMAGE ID       CREATED             SIZE
nginx-hello-world   latest    48d55fb90cc8   About a minute ago  279MB
nginx               latest    f5c017fb33c6   17 hours ago        279MB
ubuntu              latest    440dcf6a5640   3 weeks ago         117MB
PS C:\Users\Влад\docker_nginx>
```

Как видно на рисунке выше, новый образ отобразился.

Запустим новый контейнер, заранее переадресуя порт 80 на 9999.

```
PS C:\Users\Влад\docker_nginx> docker run -d -p 9999:80 nginx-hello-world
770fc647d62029612633c31ca52d71607e7fe660f77ccfa2ab93ef953fa1f079
PS C:\Users\Влад\docker_nginx>
```

```
PS C:\Users\Влад\docker_nginx> docker run -d -p 9999:80 nginx-hello-world
770fc647d62029612633c31ca52d71607e7fe660f77ccfa2ab93ef953fa1f079
PS C:\Users\Влад\docker_nginx> docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS
          NAMES
770fc647d620   nginx-hello-world  "/docker-entrypoint.…"   40 seconds ago   Up 39 seconds   0.0.0.0:9999->80/tcp, [::]:9999->
80/tcp   jovial_gagarin
365846352ed4   nginx              "/docker-entrypoint.…"   27 minutes ago   Up 27 minutes   0.0.0.0:8888->80/tcp, [::]:8888->
80/tcp   condescending_aryabhata
3e26a52ffe66   ubuntu             "/bin/bash"              34 minutes ago   Up 34 minutes
          zen_goldstine
PS C:\Users\Влад\docker_nginx>
```

Как мы видим, контейнер успешно запущен.
Перейдем в браузере на порт 9999:80.
код на html был упрощен, тк тот, что на скрине выше я косячно написал и он не работал. чтоб наверняка сделали все максимально просто))

```
index.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Hello from docker</title>
5  </head>
6  <body>
7      <h1>Hello World from nginx in docker</h1>
8  </body>
9  </html>
10
```

Перейдя на порт, увидим свой текст, а значит - мы успешно создали свой контейнер и запустили.

Hello World от nginx в Docker

В Dockerfile также можно использовать команду RUN для того, чтобы скомпилировать программу или приложение, которое мы бы сделали наподобие в сайтов в примере выше.
Также там есть команды CMD, которая является входной точкой для запуска приложения.